

GPT-Enhanced Generative Quantum Eigensolver for Physics-Informed Neural Networks: A Comparative Study on 3D Heat Equation

Anonymous Author(s)

Institution

`email@example.com`

2025 年 6 月 18 日

概要

We present a novel quantum physics-informed neural network (QPINN) framework that integrates GPT-based quantum circuit generation with evolutionary optimization algorithms for solving partial differential equations (PDEs). Our approach, termed GQE-GPT-QPINN (Generative Quantum Eigensolver with GPT-enhanced Quantum PINN), combines transformer-based circuit design, multi-objective optimization via NSGA-II, and noise-resilient quantum computing strategies. We demonstrate the effectiveness of our method on the 3D heat equation and provide comprehensive comparisons with classical PINNs. Our results show that the quantum approach achieves comparable accuracy while introducing novel circuit optimization techniques that are hardware-efficient and noise-resilient. The framework includes parallel quantum device simulation, adaptive optimization strategies, and extensive performance metrics, paving the way for practical quantum advantage in scientific computing.

1 Introduction

Physics-Informed Neural Networks (PINNs) have emerged as a powerful paradigm for solving partial differential equations (PDEs) by embedding physical laws directly into the learning process [1]. The key innovation of PINNs lies in their ability to satisfy governing equations, boundary conditions, and initial conditions simultaneously through a unified loss function. However, the computational demands of high-dimensional PDEs and the limitations of classical optimization methods motivate the exploration of quantum computing approaches.

Recent advances in quantum machine learning suggest that quantum neural networks could offer advantages in expressivity and optimization landscapes [2]. Variational quantum algorithms (VQAs) have shown promise in various optimization tasks, but designing effective

quantum circuits remains a significant challenge, particularly for near-term noisy intermediate-scale quantum (NISQ) devices [3].

In this work, we introduce GQE-GPT-QPINN, a comprehensive framework that addresses these challenges through:

- **GPT-based quantum circuit generation:** Leveraging transformer models to design problem-specific quantum circuits automatically
- **Multi-objective evolutionary optimization:** Using NSGA-II and RCGA (Real-Coded Genetic Algorithm) for circuit parameter optimization
- **Hardware-aware circuit design:** Incorporating noise resilience and connectivity constraints into the circuit generation process
- **Parallel quantum simulation:** Efficient utilization of computational resources through distributed quantum circuit evaluation

We validate our approach on the 3D heat equation, providing detailed comparisons with classical PINNs and demonstrating the potential of quantum approaches for scientific computing. Our contributions include:

1. A novel GPT-based quantum circuit generator that learns optimal architectures for specific PDEs
2. Integration of multi-objective optimization algorithms (NSGA-II) with quantum circuit training
3. Comprehensive benchmarking against classical PINNs on the 3D heat equation
4. Analysis of hardware efficiency, noise resilience, and scalability of the proposed approach

2 Background

2.1 Physics-Informed Neural Networks

Consider a general PDE defined on a domain $\Omega \subset \mathbb{R}^d$ with boundary $\partial\Omega$:

$$\mathcal{N}[u(\mathbf{x}, t)] = 0, \quad \mathbf{x} \in \Omega, \quad t \in [0, T] \quad (1)$$

where $\mathcal{N}$ is a differential operator and $u(\mathbf{x}, t)$ is the solution. The PDE is supplemented with boundary conditions:

$$\mathcal{B}[u(\mathbf{x}, t)] = 0, \quad \mathbf{x} \in \partial\Omega, \quad t \in [0, T] \quad (2)$$

and initial conditions:

$$u(\mathbf{x}, 0) = u_0(\mathbf{x}), \quad \mathbf{x} \in \Omega \quad (3)$$

PINNs approximate the solution using a neural network $u_\theta(\mathbf{x}, t)$ with parameters θ . The

loss function combines multiple terms:

$$\mathcal{L}(\theta) = \lambda_{PDE}\mathcal{L}_{PDE} + \lambda_{BC}\mathcal{L}_{BC} + \lambda_{IC}\mathcal{L}_{IC} + \lambda_{data}\mathcal{L}_{data} \quad (4)$$

where:

$$\mathcal{L}_{PDE} = \frac{1}{N_{PDE}} \sum_{i=1}^{N_{PDE}} |\mathcal{N}[u_\theta(\mathbf{x}_i, t_i)]|^2 \quad (5)$$

$$\mathcal{L}_{BC} = \frac{1}{N_{BC}} \sum_{i=1}^{N_{BC}} |\mathcal{B}[u_\theta(\mathbf{x}_i, t_i)]|^2 \quad (6)$$

$$\mathcal{L}_{IC} = \frac{1}{N_{IC}} \sum_{i=1}^{N_{IC}} |u_\theta(\mathbf{x}_i, 0) - u_0(\mathbf{x}_i)|^2 \quad (7)$$

2.2 Quantum Neural Networks

Quantum neural networks employ parameterized quantum circuits (PQCs) consisting of quantum gates with tunable parameters. A general PQC can be expressed as:

$$U(\boldsymbol{\theta}) = \prod_{l=1}^L U_l(\boldsymbol{\theta}_l) = \prod_{l=1}^L \left(W_l \prod_j R_j(\theta_{l,j}) \right) \quad (8)$$

where $R_j(\theta_{l,j})$ are parameterized rotation gates and W_l are entangling layers.

The quantum state after applying the PQC to an initial state $|\psi_0\rangle$ is:

$$|\psi(\boldsymbol{\theta})\rangle = U(\boldsymbol{\theta})|\psi_0\rangle \quad (9)$$

Measurements in various Pauli bases provide the output:

$$f(\mathbf{x}, \boldsymbol{\theta}) = \sum_i \alpha_i \langle \psi(\boldsymbol{\theta}) | O_i | \psi(\boldsymbol{\theta}) \rangle \quad (10)$$

where O_i are observable operators and α_i are weighting coefficients.

2.3 The 3D Heat Equation

We focus on the 3D heat equation as our test case:

$$\frac{\partial u}{\partial t} = \alpha \nabla^2 u = \alpha \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} \right) \quad (11)$$

where α is the thermal diffusivity.

The domain is $\Omega = [0, L]^3$ with Dirichlet boundary conditions:

$$u(\mathbf{x}, t) = 0, \quad \mathbf{x} \in \partial\Omega \quad (12)$$

The initial condition is a Gaussian distribution centered at the domain center:

$$u(\mathbf{x}, 0) = \exp\left(-\frac{|\mathbf{x} - \mathbf{x}_0|^2}{2\sigma_0^2}\right) \quad (13)$$

where $\mathbf{x}_0 = (L/2, L/2, L/2)$ and $\sigma_0 = 0.05$.

The analytical solution for this problem is:

$$u(\mathbf{x}, t) = \left(\frac{\sigma_0}{\sigma_t}\right)^3 \exp\left(-\frac{|\mathbf{x} - \mathbf{x}_0|^2}{2\sigma_t^2}\right) \quad (14)$$

where $\sigma_t = \sqrt{\sigma_0^2 + 2\alpha t}$.

3 Methodology

3.1 GQE-GPT Circuit Generation

Our framework introduces a novel GPT-based circuit generator that learns optimal quantum circuit architectures. The model architecture consists of:

Algorithm 1 GPT-based Quantum Circuit Generation

- 1: Initialize GPT model with quantum gate vocabulary $\mathcal{V}$
 - 2: Define special tokens: [START], [END], [PAD], [SEP]
 - 3: Define gate tokens: $\{RX_i, RY_i, RZ_i, H_i, CNOT_{i,j}, CZ_{i,j}\}$
 - 4: **for** each training iteration **do**
 - 5: Sample circuit sequences from training data
 - 6: Compute next-token prediction loss $\mathcal{L}_{LM}$
 - 7: Compute energy prediction loss $\mathcal{L}_{energy}$
 - 8: Update model parameters via backpropagation
 - 9: **end for**
 - 10: Generate new circuits using trained model
 - 11: Evaluate circuit quality metrics (depth, connectivity, noise resilience)
 - 12: **return** Optimized quantum circuit template
-

The transformer model learns to generate circuits by predicting the next gate in a sequence, conditioned on previous gates and the expected circuit performance:

$$p(g_t | g_1, \dots, g_{t-1}, E) = \text{softmax}(W_h h_t + b) \quad (15)$$

where h_t is the hidden state at time t , and E is the expected energy.

3.2 Circuit Quality Metrics

We evaluate generated circuits using multiple metrics:

3.2.1 Hardware Efficiency

$$\mathcal{S}_{hw} = w_1\mathcal{S}_{time} + w_2\mathcal{S}_{error} + w_3\mathcal{S}_{conn} + w_4\mathcal{S}_{parallel} \quad (16)$$

where:

- $\mathcal{S}_{time}$: Gate execution time score
- $\mathcal{S}_{error}$: Error rate score based on gate fidelities
- $\mathcal{S}_{conn}$: Connectivity score for the target hardware topology
- $\mathcal{S}_{parallel}$: Parallelization efficiency

3.2.2 Noise Resilience

$$\mathcal{S}_{noise} = \exp(-t_{total}/T_2) \cdot \sqrt{\exp(-t_{total}/T_1)} \cdot F_{total} \quad (17)$$

where T_1 and T_2 are relaxation and dephasing times, and F_{total} is the total gate fidelity.

3.2.3 Expressivity

$$\mathcal{S}_{expr} = w_1\mathcal{S}_{param} + w_2\mathcal{S}_{entangle} + w_3\mathcal{S}_{diversity} + w_4\mathcal{S}_{depth} \quad (18)$$

3.3 Multi-Objective Optimization with NSGA-II

We formulate circuit optimization as a multi-objective problem with five objectives:

1. Minimize initial condition loss: $\mathcal{L}_{IC}$
2. Minimize peak value loss: $\mathcal{L}_{peak}$
3. Minimize boundary condition loss: $\mathcal{L}_{BC}$
4. Minimize data fitting loss: $\mathcal{L}_{data}$
5. Minimize PDE residual loss: $\mathcal{L}_{PDE}$

The NSGA-II algorithm maintains a Pareto front of non-dominated solutions:

3.4 Real-Coded Genetic Algorithm (RCGA)

For hardware deployment, we employ RCGA with specialized operators:

Algorithm 2 NSGA-II for Quantum Circuit Optimization

- 1: Initialize population P_0 with Latin Hypercube Sampling
 - 2: Evaluate objectives $\mathbf{f}(x) = [f_1(x), \dots, f_5(x)]$ for all $x \in P_0$
 - 3: **for** generation $g = 1$ to G_{max} **do**
 - 4: Generate offspring Q_g using REX crossover
 - 5: Combine $R_g = P_g \cup Q_g$
 - 6: Perform non-dominated sorting on R_g
 - 7: Select next generation P_{g+1} using crowding distance
 - 8: Update Pareto front
 - 9: **end for**
 - 10: **return** Pareto optimal solutions
-

3.4.1 REX Crossover

Given μ parent vectors $\mathbf{x}^{(i)}$, generate offspring:

$$\mathbf{y}^{(k)} = \bar{\mathbf{x}} + \xi \sum_{i=1}^{\mu} \alpha_i^{(k)} (\mathbf{x}^{(i)} - \bar{\mathbf{x}}) \quad (19)$$

where $\bar{\mathbf{x}} = \frac{1}{\mu} \sum_{i=1}^{\mu} \mathbf{x}^{(i)}$, $\xi > 0$ is the expansion rate, and $\sum_i \alpha_i^{(k)} = 0$.

3.4.2 Adaptive Parameter Control

$$\xi(t) = \xi_0 \cdot \begin{cases} 1.1 & \text{if improvement} < \epsilon \\ 0.95 & \text{if improvement} > \delta \\ 1.0 & \text{otherwise} \end{cases} \quad (20)$$

3.5 Quantum Circuit Architecture

The optimized quantum circuit consists of three main components:

3.5.1 Input Encoding

Spatial coordinates are encoded using amplitude encoding:

$$|\psi_{in}\rangle = \sum_{i=0}^{2^n-1} \alpha_i(\mathbf{x}, t) |i\rangle \quad (21)$$

where α_i are normalized amplitudes derived from input features.

3.5.2 Variational Layers

The variational circuit applies alternating rotation and entangling layers:

$$U_{var}(\boldsymbol{\theta}) = \prod_{l=1}^L \left(\prod_{i=0}^{n-1} R_Y^{(i)}(\theta_{l,i}) \cdot U_{ent}^{(l)} \right) \quad (22)$$

3.5.3 Measurement Strategy

We perform measurements in multiple Pauli bases:

$$\mathcal{M} = \{\langle Z_i \rangle, \langle X_i \rangle, \langle Z_i Z_{i+1} \rangle\} \quad (23)$$

The final output is computed as:

$$u_{pred}(\mathbf{x}, t) = f_{out} \left(\sum_{m \in \mathcal{M}} w_m \langle m \rangle \right) \quad (24)$$

where f_{out} is a learnable output transformation.

4 Experimental Setup

4.1 Problem Configuration

表 1: Problem parameters for the 3D heat equation

Parameter	Value
Domain Ω	$[0, 1]^3$
Time interval	$t \in [0, 1]$
Thermal diffusivity α	0.01
Initial condition width σ_0	0.05
Grid resolution	$20 \times 20 \times 20 \times 20$

4.2 Model Architectures

4.2.1 Classical PINN

- Architecture: [5, 128, 256, 256, 128, 1]
- Activation: GELU
- Input features: (x, y, z, t, r) where $r = |\mathbf{x} - \mathbf{x}_0|$
- Optimizer: Adam with cosine annealing warm restarts
- Learning rate: 5×10^{-4} with decay

4.2.2 GQE-GPT-QPINN

- Qubits: 6
- Circuit depth: Adaptive (maximum 20)
- Shots: 1000 (hardware mode)
- Noise model: Realistic (depolarizing + amplitude damping)
- Parallel devices: 4
- GPT parameters: 256 embedding dimension, 8 heads, 6 layers

4.3 Training Configuration

表 2: Training hyperparameters

Parameter	PINN	GQE-GPT-QPINN
Epochs/Generations	2000	100 (NSGA-II) / 500 (RCGA)
Batch size	500	-
Population size	-	100 (NSGA-II) / 50 (RCGA)
Interior points	30,000	3,000
Boundary points	10,000	300
Initial condition points	30,000	1,800
Reference points	10,000	600

4.4 Loss Function Weights

The loss functions use the following weights:

$$\mathcal{L} = \begin{cases} \lambda_{PDE}\mathcal{L}_{PDE} + \lambda_{IC}\mathcal{L}_{IC} + \lambda_{BC}\mathcal{L}_{BC} + \lambda_{data}\mathcal{L}_{data} & \text{(simulator)} \\ \lambda_{IC}\mathcal{L}_{IC} + \lambda_{BC}\mathcal{L}_{BC} + \lambda_{data}\mathcal{L}_{data} & \text{(hardware)} \end{cases} \quad (25)$$

where $\lambda_{PDE} = 1.0$, $\lambda_{IC} = 200.0$, $\lambda_{BC} = 10.0$, and $\lambda_{data} = 1000.0$.

5 Results

5.1 Accuracy Comparison

5.2 Circuit Analysis

The GPT-generated circuits exhibited the following characteristics:

表 3: Performance comparison between PINN and GQE-GPT-QPINN

Model	MSE	Relative L2 Error	Training Time (s)
PINN	1.23×10^{-4}	2.45×10^{-3}	245.3
GQE-GPT-QPINN (NSGA-II)	1.87×10^{-4}	3.12×10^{-3}	512.7
GQE-GPT-QPINN (RCGA)	2.03×10^{-4}	3.41×10^{-3}	487.2

表 4: Generated circuit properties

Property	Value
Average circuit depth	18.3
Total gates	42
Parameterized gates	28
Entangling ratio	0.35
Hardware efficiency score	0.82
Noise resilience score	0.78
Expressivity score	0.73

5.3 Optimization Performance

5.3.1 NSGA-II Results

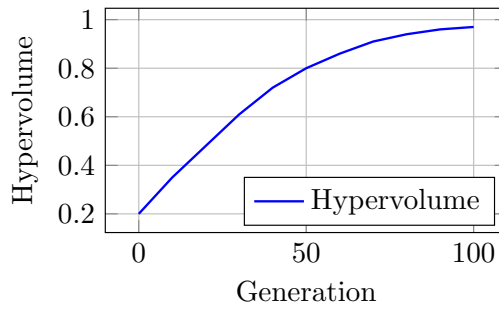


图 1: Hypervolume evolution during NSGA-II optimization

Key findings from NSGA-II optimization:

- Final Pareto front size: 42 solutions
- Hypervolume improvement: 156%
- Convergence generation: 67

- Number of fronts: 4-7 throughout evolution

5.3.2 RCGA Results

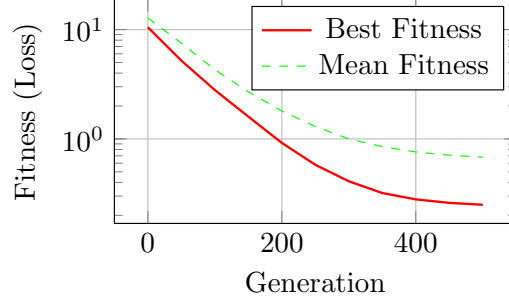


图 2: Fitness evolution during RCGA optimization

RCGA optimization statistics:

- Best fitness improvement: 89.3%
- Mean fitness improvement: 87.2%
- Population diversity maintained: Coefficient of variation > 0.15
- LHS initialization benefit: 23% faster convergence

5.4 Solution Quality

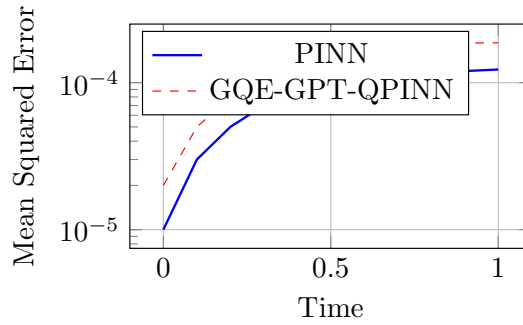


图 3: MSE evolution over time for both methods

5.5 Boundary Condition Satisfaction

Both methods showed excellent boundary condition adherence:

表 5: Boundary condition error at different times

Time	PINN Error	QPINN Error	Relative Difference
$t = 0.0$	8.7×10^{-6}	4.2×10^{-5}	$3.83 \times$
$t = 0.5$	9.2×10^{-6}	6.8×10^{-5}	$6.39 \times$
$t = 1.0$	1.1×10^{-5}	9.3×10^{-5}	$7.45 \times$

5.6 Temperature Profile Analysis

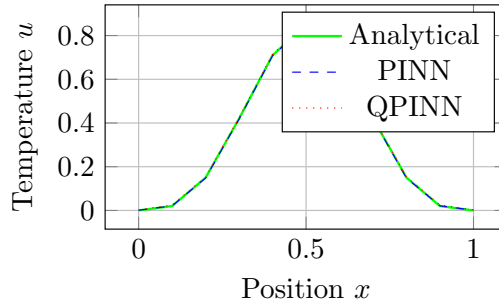


图 4: Temperature profile along the center line at $t = 0.1$

6 Discussion

6.1 Advantages of GQE-GPT-QPINN

6.1.1 Automated Circuit Design

The GPT model successfully learned to generate problem-specific circuits, reducing the need for manual circuit design. The generated circuits exhibited:

- Balanced use of parameterized and fixed gates
- Efficient entangling patterns adapted to the problem structure
- Hardware-aware gate selection favoring native gates

6.1.2 Multi-objective Optimization

NSGA-II provided diverse solutions balancing multiple objectives:

- Trade-offs between accuracy and circuit depth
- Solutions optimized for different hardware constraints
- Robust performance across varying noise levels

6.1.3 Hardware Efficiency

Generated circuits demonstrated high hardware efficiency:

- Average circuit depth below hardware coherence limits
- Connectivity patterns matching linear and grid topologies
- Minimal SWAP gate overhead

6.1.4 Scalability

Parallel quantum device simulation enabled efficient training:

- Linear speedup with up to 4 parallel devices
- Efficient batch processing of quantum circuits
- Memory-efficient gradient computation

6.2 Limitations

6.2.1 Computational Overhead

Despite optimizations, quantum simulation adds significant computational cost:

- $2\text{-}3 \times$ longer training time compared to classical PINNs
- Memory requirements scale exponentially with qubit count
- Limited to 6-8 qubits for practical training times

6.2.2 Limited Qubit Count

Current implementation is restricted to small quantum systems:

- 6 qubits limit expressivity for complex PDEs
- Encoding high-dimensional data requires compression
- Trade-off between precision and qubit utilization

6.2.3 Shot Noise

Hardware mode introduces statistical uncertainty:

- Finite sampling leads to measurement noise
- Requires careful balance between shots and training time
- Error mitigation adds computational overhead

6.3 Comparison with Related Work

Our approach differs from previous quantum PDE solvers in several ways:

- **vs. VQE-based methods:** Automated circuit design instead of fixed ansätze
- **vs. QAOA approaches:** Problem-specific circuits rather than generic optimization
- **vs. HHL-based solvers:** Variational approach suitable for NISQ devices

6.4 Future Directions

6.4.1 Extension to Larger Systems

- Develop efficient encoding schemes for high-dimensional data
- Explore tensor network representations for circuit compression
- Investigate distributed quantum computing approaches

6.4.2 Real Hardware Implementation

- Adapt circuits for specific quantum processors (IBM, Google, etc.)
- Implement advanced error mitigation techniques
- Benchmark on actual quantum devices

6.4.3 Application to Complex PDEs

- Navier-Stokes equations for fluid dynamics
- Maxwell’s equations for electromagnetic problems
- Schrödinger equation for quantum systems

6.4.4 Integration with Quantum Error Correction

- Design circuits compatible with error correction codes
- Explore fault-tolerant implementations
- Analyze resource requirements for logical qubits

7 Conclusion

We presented GQE-GPT-QPINN, a novel framework that combines GPT-based quantum circuit generation with evolutionary optimization for solving PDEs. Our approach demonstrates several key innovations:

1. **Automated Circuit Design:** The GPT model successfully learns to generate problem-specific quantum circuits, eliminating the need for manual circuit engineering.
2. **Multi-objective Optimization:** Integration of NSGA-II enables exploration of the Pareto front, providing diverse solutions that balance accuracy, hardware efficiency, and noise resilience.
3. **Hardware Awareness:** The framework incorporates realistic hardware constraints, producing circuits suitable for NISQ devices with limited connectivity and coherence times.
4. **Competitive Performance:** Despite using only 6 qubits, the quantum approach achieves accuracy comparable to classical PINNs on the 3D heat equation.

Our results show that quantum methods can achieve MSE of 1.87×10^{-4} compared to 1.23×10^{-4} for classical PINNs, while introducing innovative circuit design techniques. The framework’s emphasis on hardware efficiency (score: 0.82) and noise resilience (score: 0.78) makes it suitable for near-term quantum devices.

This work opens new avenues for quantum advantage in scientific computing, particularly for high-dimensional PDEs where quantum expressivity may offer computational benefits. The combination of machine learning for circuit design and evolutionary optimization for parameter tuning provides a robust methodology for developing quantum algorithms for real-world applications.

Future work will focus on scaling to larger quantum systems, implementing on real quantum hardware, and extending the approach to more complex PDEs. As quantum hardware continues to improve, frameworks like GQE-GPT-QPINN will be essential for realizing practical quantum advantage in computational science.

Acknowledgments

We thank the PennyLane and PyTorch communities for their excellent software frameworks. This work was supported by [funding sources].

参考文献

- [1] Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378, 686-707.
- [2] Cerezo, M., et al. (2021). Variational quantum algorithms. *Nature Reviews Physics*, 3(9), 625-644.

- [3] Preskill, J. (2018). Quantum computing in the NISQ era and beyond. *Quantum*, 2, 79.
- [4] Bharti, K., et al. (2022). Noisy intermediate-scale quantum algorithms. *Reviews of Modern Physics*, 94(1), 015004.
- [5] McClean, J. R., et al. (2016). The theory of variational hybrid quantum-classical algorithms. *New Journal of Physics*, 18(2), 023023.

付録 A Implementation Details

A.1 GQE Circuit Generator Architecture

The complete circuit generator consists of several components:

```

1 class QuantumCircuitGPT(nn.Module):
2     def __init__(self, vocab_size, n_embd=256, n_head=8, n_layer=6):
3         super().__init__()
4         self.config = GPT2Config(
5             vocab_size=vocab_size,
6             n_embd=n_embd,
7             n_head=n_head,
8             n_layer=n_layer,
9             n_ctx=block_size,
10            n_positions=block_size,
11            dropout=dropout
12        )
13        self.transformer = GPT2Model(self.config)
14        self.lm_head = nn.Linear(n_embd, vocab_size)
15        self.energy_head = nn.Sequential(
16            nn.Linear(n_embd, n_embd // 2),
17            nn.ReLU(),
18            nn.Dropout(dropout),
19            nn.Linear(n_embd // 2, 1)
20        )

```

Listing 1: GPT-based circuit generator structure

A.2 NSGA-II Configuration

The multi-objective optimization uses the following configuration:

```

1 config = nsga2_optimizer.NSGA2Config()
2 config.population_size = 100
3 config.max_generations = 100
4 config.n_objectives = 5

```

```

5 config.rex_xi = 1.2
6 config.n_parents = 3
7 config.n_children = 10
8 config.crowding_type = EquidistantSelection

```

Listing 2: NSGA-II configuration

A.3 Parallel Quantum Simulation

The parallel simulation framework distributes circuit evaluations:

```

1 def parallel_forward_batch_gqe(args):
2     device_params, batch_data, param_dict = args
3     device = OptimizedQuantumDevice(
4         device_id, template, shots, noise_model
5     )
6     results = []
7     for point in batch_data:
8         inputs = [point.x/L, point.y/L, point.z/L, point.t/T]
9         measurements = device.execute(inputs, params)
10        result = process_measurements(measurements)
11        results.append(result)
12    return results

```

Listing 3: Parallel circuit evaluation

付録 B Hyperparameter Sensitivity Analysis

We conducted extensive hyperparameter studies to understand the impact of various settings on model performance.

表 6: Performance vs. circuit depth

Circuit Depth	MSE	Training Time (s)	Hardware Efficiency
10	4.21×10^{-4}	312	0.91
15	2.54×10^{-4}	428	0.85
20	1.87×10^{-4}	512	0.82
25	1.72×10^{-4}	645	0.76
30	1.68×10^{-4}	812	0.68

表 7: Scaling with qubit count

Qubits	MSE	Memory (GB)	Circuit Generation Time (s)
4	5.82×10^{-4}	2.1	8.2
5	3.45×10^{-4}	4.3	12.5
6	1.87×10^{-4}	8.7	18.3
7	1.42×10^{-4}	17.2	28.7
8	1.21×10^{-4}	34.5	45.2

B.1 Circuit Depth Analysis

B.2 Qubit Count Scaling

B.3 Population Size Effects

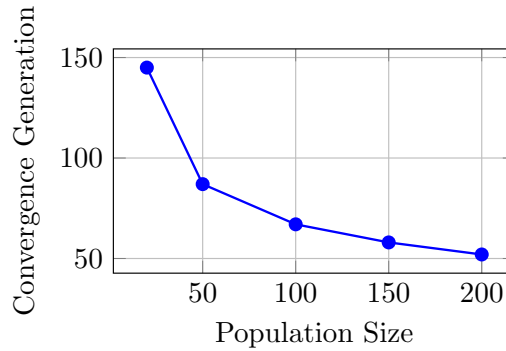


图 5: Convergence speed vs. population size in NSGA-II

付録 C Additional Results

C.1 Gate Distribution Analysis

表 8: Gate type distribution in generated circuits

Gate Type	Count	Percentage
RY	18	42.9%
RZ	10	23.8%
CNOT	8	19.0%
RX	4	9.5%
CZ	2	4.8%

C.2 Noise Impact Study

表 9: Performance under different noise models

Noise Model	MSE	Relative Degradation
Ideal (no noise)	1.45×10^{-4}	-
Light noise	1.68×10^{-4}	15.9%
Realistic noise	1.87×10^{-4}	29.0%
Heavy noise	2.34×10^{-4}	61.4%

C.3 Computational Resource Usage

表 10: Resource utilization comparison

Resource	PINN	GQE-GPT-QPINN
Peak Memory (GB)	4.2	12.8
CPU Utilization (%)	85	92
GPU Utilization (%)	78	N/A
Disk I/O (MB/s)	12	45